Algorithm

Dynamic Programming

knapsack 0/1 (for things that we cannot take fraction)

sack limit $W = 70$

weight $w = \{10, 30, 40, 50\}$

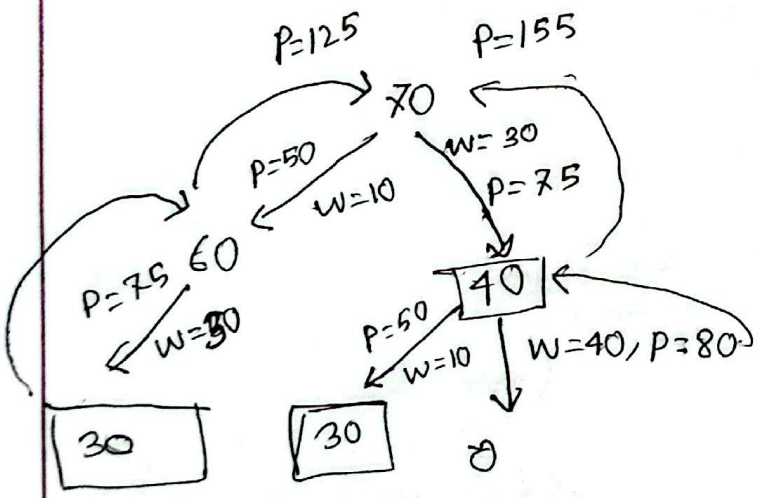
profit $v = \{50, 75, 80, 100\}$

$\frac{v}{w} = 5, 2.5, 2, 2$

$w = 10 + 30$

$p = 50 + 75 = 125$

} Greedy approach in 0/1 case gives me sub-optimal answer



recursion

$O(2^n)$

dp

$O(2n)$ → depends on capacity and combination

i

capacity * i

```
knapSack01 ( capacity, i, w, v ) {
    if (i == n) return 0.
    if(dp[i][capacity] != -1) return dp[i][capacity]
    int a = knapsack01 (capacity, i+1, w, v);
    if ( capacity ≥ w[i])
    int b = knapsack01 (capacity - w[i], i+1, w, v);
    dp [i][capacity] = max (a, b);
    return max(a, b);
}
```

not considering the ith item ←

considering the ith item ←

dp[i][capacity] → from 0 to ith item with capacity what is the max profit

capacity = 70

w = 40

remaining = 30

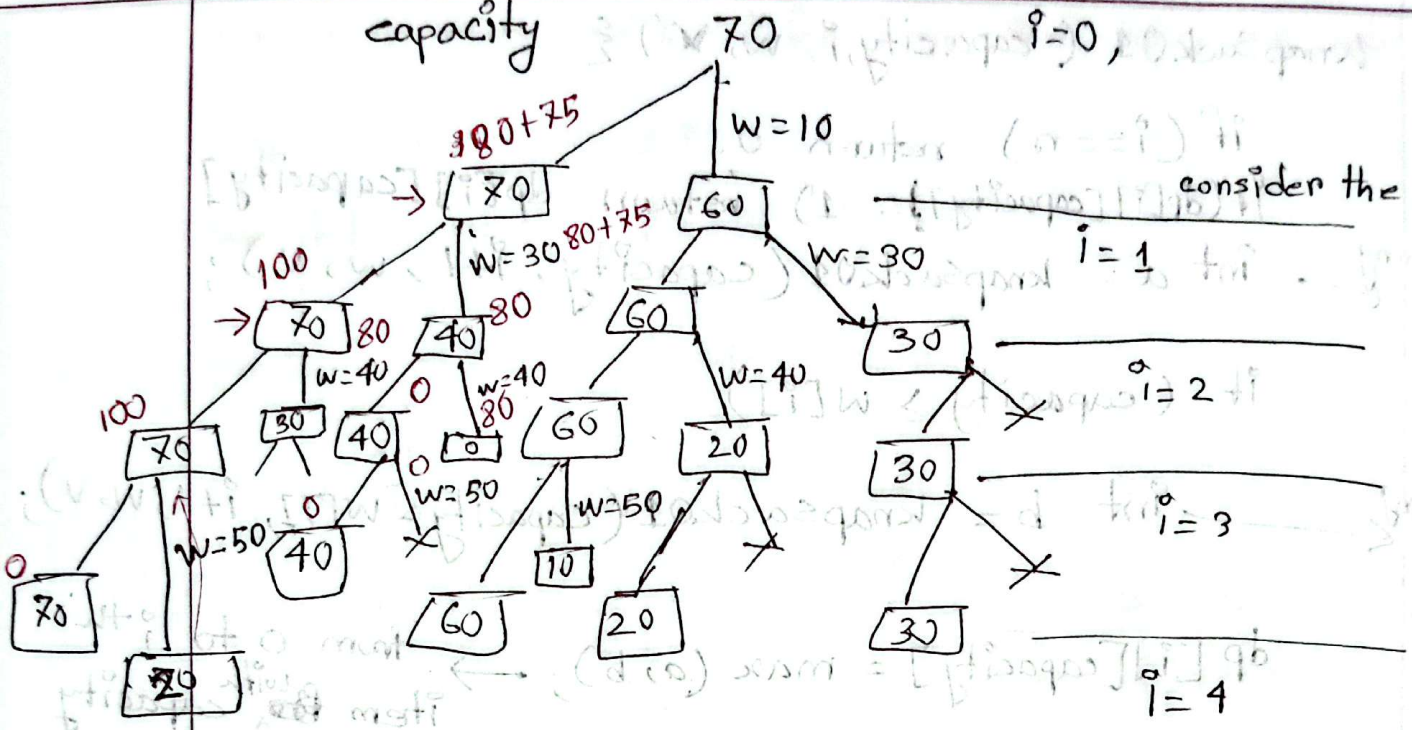
vector<int> vec[i], i's

= knapsack (30, 1, w, v)

a = knapsack (30, 2, w, v)

b = knapsack (0, 2, w, v)

dp[1][30] = max(a, b)



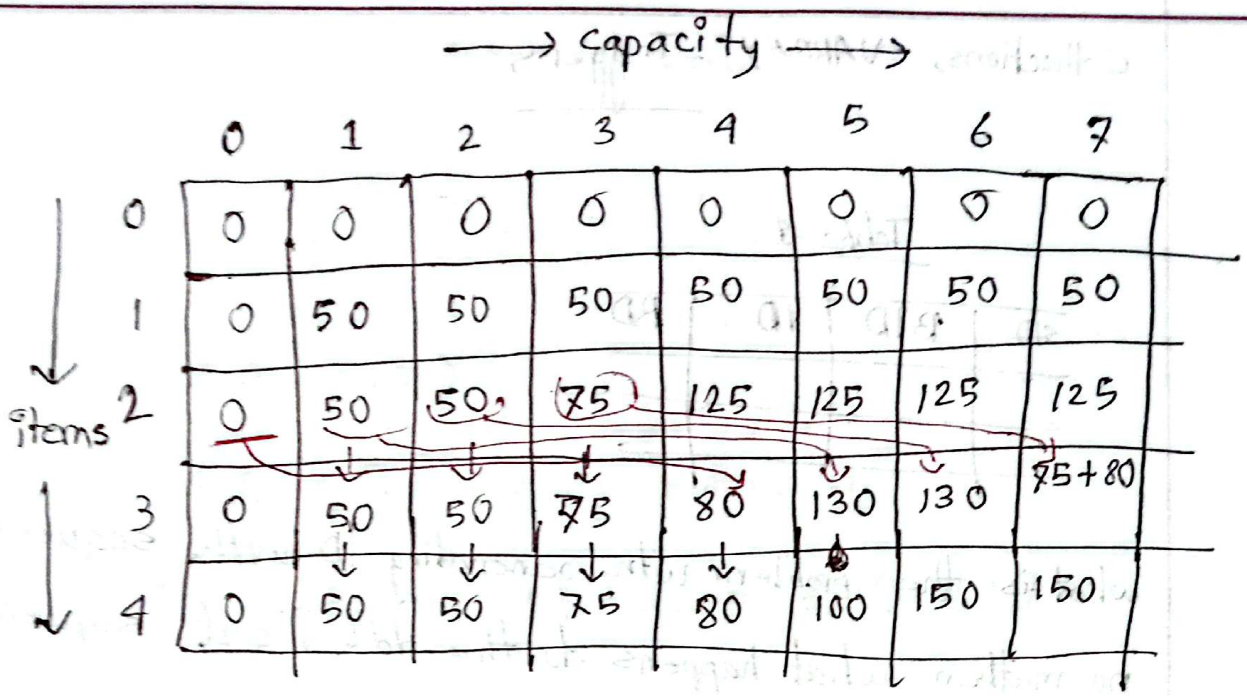
Iterative Approach

~~vv=0~~; $W = 7$

$w = \{1, 3, 4, 5\}$

$v = \{50, 75, 80, 100\}$

$dp[c - w[i]][i-1] + v[i]$


→ capacity →
items
0 1 2 3 4 5 6 7
0 0 0 0 0 0 0 0 0
1 0 50 50 50 50 50 50 50
2 0 50 50 75 125 125 125 125
3 0 50 50 75 80 130 130 75+80
4 0 50 50 75 80 100 150 150

Algorithm

Binomial Coefficient

$$^{n}C_{r} = {}^{n-1}C_{r} + {}^{n-1}C_{r-1}$$

```
int ncr( n, r ) {
    if ( n == 0 || r = 0 return 1
    if dp[n][r] = -1 return dp[n][r]
    return dp[n][r] = ncr(n-1, r) + ncr(n-1, r-1);
```

Iterative version:

```
for (n=0; n≤N; n++) {
    for (r=0; r≤R; r++) {
        if (n==0 || r==0) dp[n][r] = 1;
        else
            dp[n][r] = dp[n-1][r] + dp[n-1][r-1];
return dp[n][r];
```

Minimum Edit distance

x = "aabab", y = "babab"

Or transform y into x by following operation

1. Deletion from x, $Del(x_i) = 1$
2. Insertion y_j into x, $Ins(y_j) = 1$
3. change / replace x_i with y_j, $c(x_i, y_j) = 2$

Minimize the total cost?

0	1	2	3	4
a	a	b	a	b

0	1	2	3	4
b	a	b	a	b

del (x_2) = a a a b

ins $(y_1, 1)$

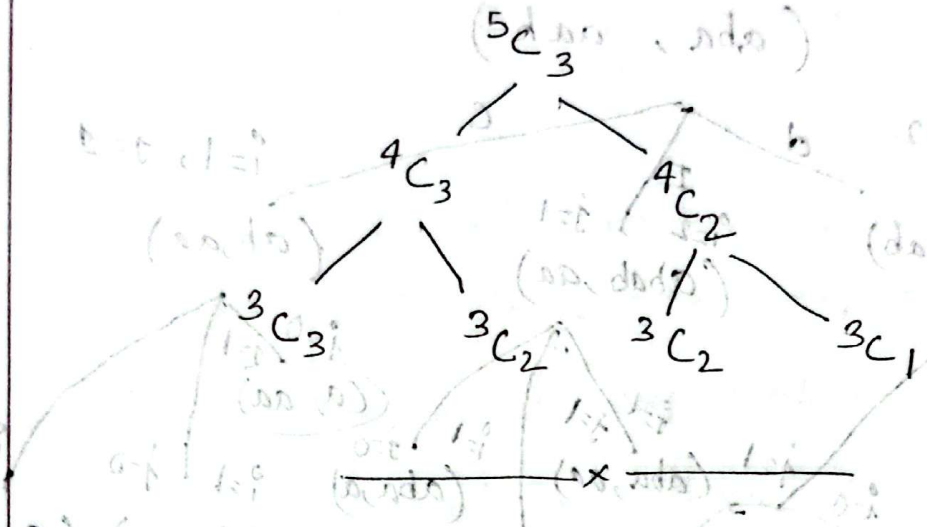
$\Rightarrow$ a a a b a b

change / replace

change (x_3, y_4)

= a a b b b

for co-eff




```
int editDistance ( i, j ) {
    if ( i == -1) return j+1;
    if ( j == -1) return i+1;
    if (xi == yj)
        editdistance(i-1, j-1).
```

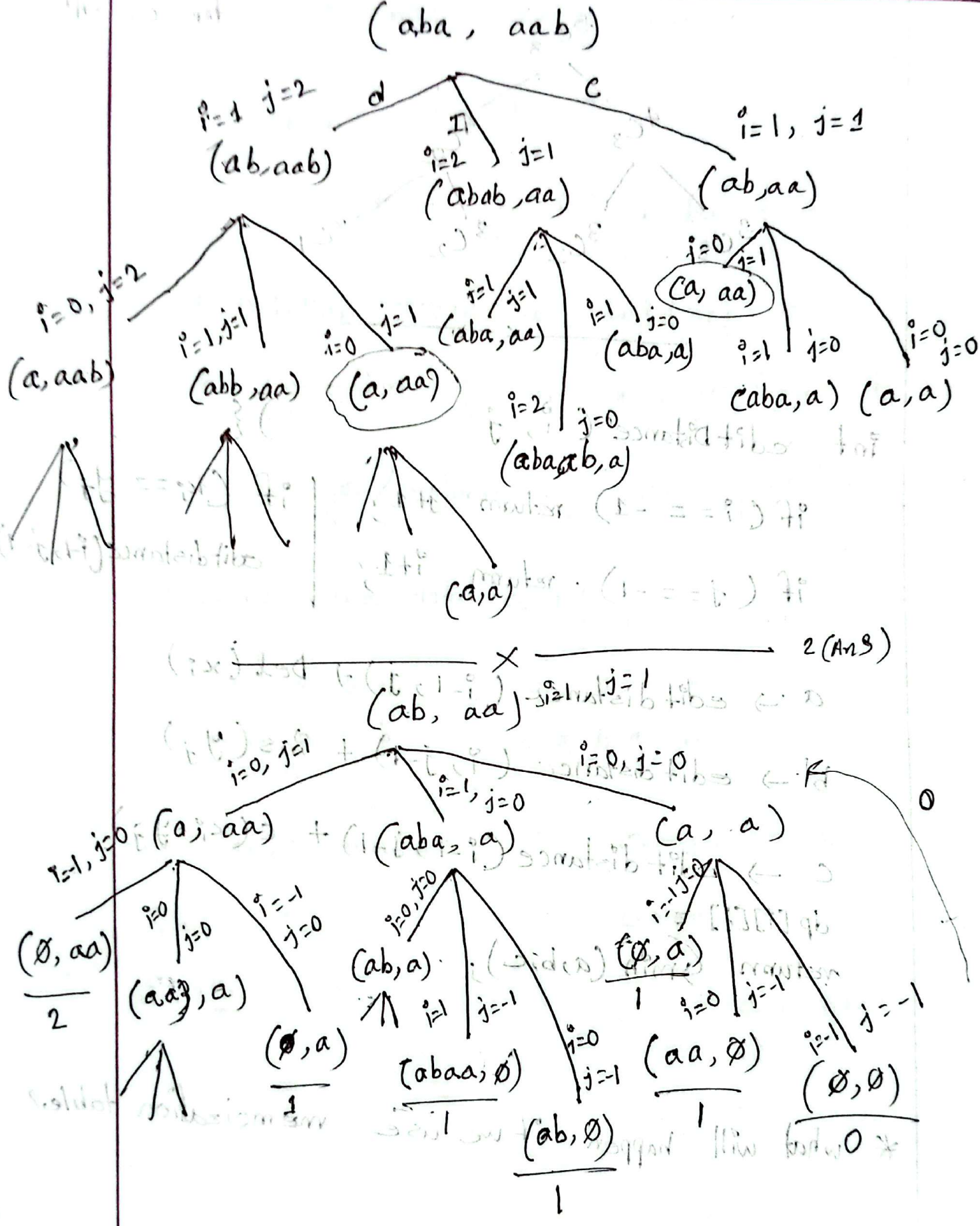
a → editdistance (i-1, j) + Del (xi)

b → editdistance (i, j-1) + ins (yj)

c → editdistance (i-1, j-1) + c(xi, yj).

```
dp[i][j] =
return (min (a,b,c));
```

* what will happen if we use memoization table?


(aba, aab)
i=1 j=2
(ab, aab)
I
i=2 j=1
(abab, aa)
C
i=1, j=1
(ab, aa)
i=0, j=2
(a, aab)
i=1, j=1
(abb, aa)
i=0 j=1
(a, aa)
(aba, aa)
(aba, a)
(a, aa)
(aba, a)
(a, a)
(aba, b, a)
(a, a)
2 (Ans)
(ab, aa)
i=0, j=1
(a, aa)
i=1, j=0
(aba, a)
i=0, j=0
(a, a)
0
(ø, aa)
2
(aa, a)
(ø, a)
1
(ab, a)
(abaa, ø)
1
(ab, ø)
1
(ø, a)
1
(aa, ø)
1
(ø, ø)
0

Algorithm

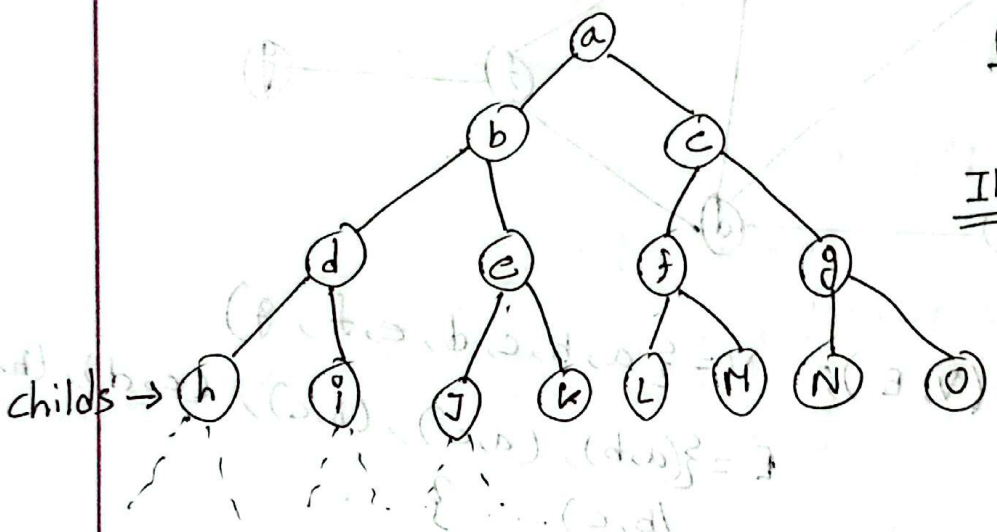
Basic Traversal and search

Binary tree traversal:

① Pre Order — D, L, R

② Inorder — L, D, R

③ Post Order — L, R, D



Pre: a, b, d, h, i, e, J, k, c, f, L, M, g, N, O

IN: h, d, i, b, J, e, k, a, L, f, M, c, N, g, O

post: h, i, d, J, k, e, b, L, M, f, N, O, g, C, a

sequence of processing

L → go to left

D → process the current node → print(cur)

R → go to right

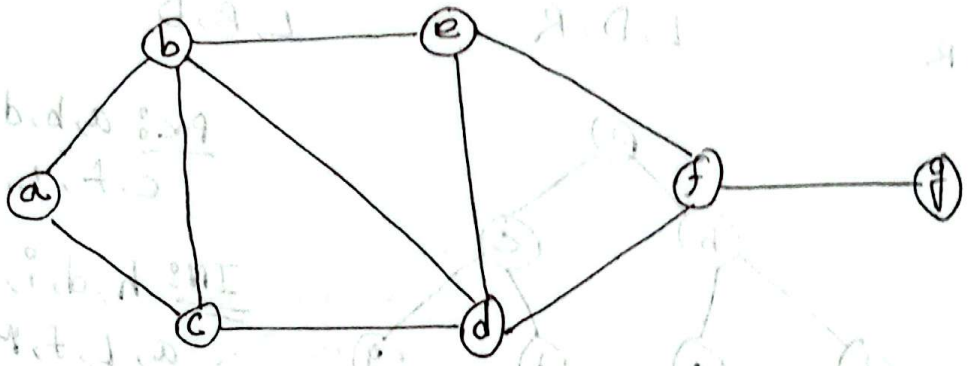
- L, D, R
- L, R, D
- D, L, R
- D, R, L
- R, D, L
- R, L, D

A graph would be a Tree if : no circle & (n-1) edges.

Graph Traversal or Search

i) Breadth First Search (BFS)

ii) Depth First search (DFS)



Graph : $G(V, E)$; $V = \{a, b, c, d, e, f, g\}$

$E = \{(a,b), (a,c), (b,c), (c,d), (b,d), (b,e) \ldots\}$

Some Terminology :

visit (v) : you have seen the node v while searching/Traversing

Explored (v) : you have gone through all the adjacent edges of node v.

O(V+E)

BFS

```
Algorithm BFS ( n, ~~adjus adli~~ adjlist )
    visit [n] := Initialized to false
    queueExplore := ∅
    queueExplore . push (root);
      visite [root] = true;
    while queueExplore not Empty {
        cur := queueExplore . pop( );
        ~~visit [cur] = True;~~
        for adjnode in adjlist [cur] {
            if ( visite [adjnode] = false)
              visit [adjnode] = True;
            queueExplore . push (adjnode);
        }
```

what is the issue here?

queueExplore : ⓐ ⓒ ⓑ ⓓ ⓔ ⓕ ⓖ

Time complexity : O(V+E)

space complexity : O(V)

(number of node)

In case of adjacent map List Time complexity would be $O(n^2)$

Algorithm DFS (n, adjList) {

visit [cur] = true;

for adjNode in adjList [cur] {

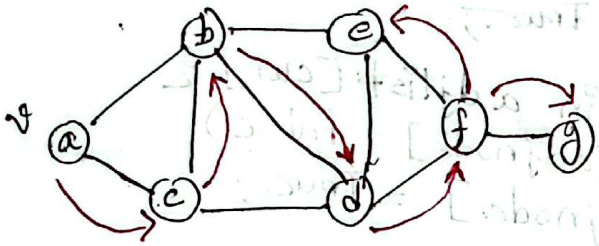
visit [adjNode] = true; ✗

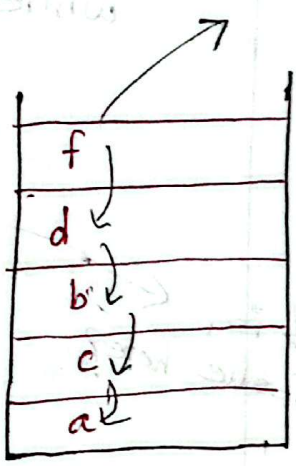
if(visit [adjNode] = false)

DFS (adjNode , adjList);

}

?



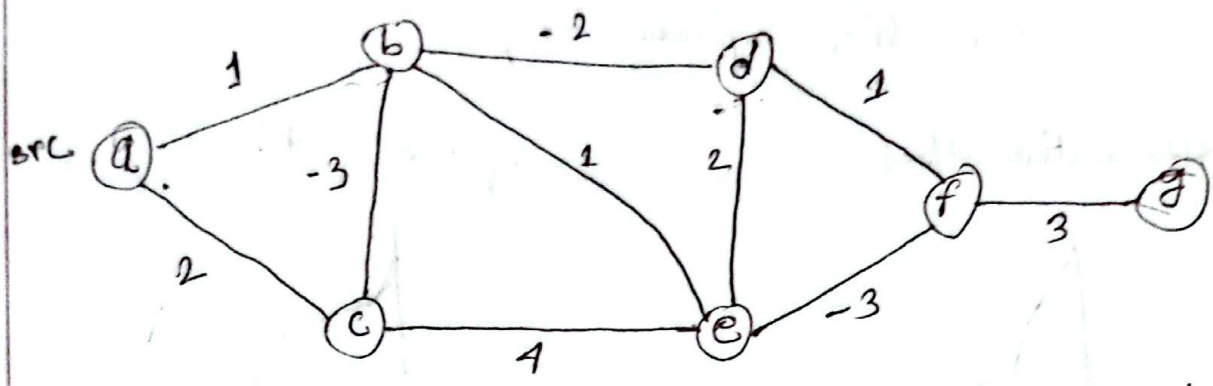


Time complexity : O (V+E)

space " : O (V)

Algorithm

Bellman - Ford (sssp)



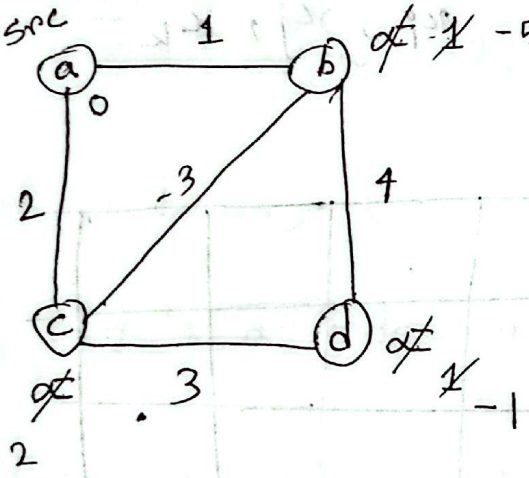
u	v	
a	b	1
a	c	2
b	c	-3
b	d	-2
b	e	1
c	e	4
d	e	2
d	f	1
e	f	-3
f	g	3

```
Algorithm BellmanFord(src, edgList){
    dist[1...n] = ∞
    dist[src] = 0;
    for(i = 1 to n-1){
        for(edge : edg list){
            if(dist[edge.u] < ∞){
                dist[edge.v] = min(dist[edge.v],
                                   dist[edge.u] + edge.w);
            }
        }
    }
    .
    .
    .
P.T.V
```

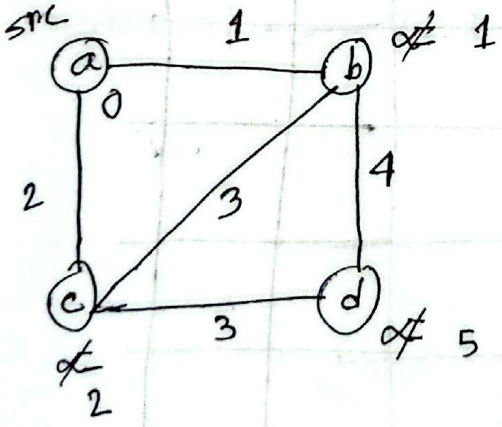
edge.u
edge.v
edge.w
from u to v we have a path with weight w

we can check if any update was happened there or not; then if no update occurs can get out of the loop.

```
for (edge : edglist) {
    if (dist[edge.u] < ∞){
        if ( dist [edge.v] < dist [edge.u] + edge. w){
            print ( Negative cycle detected with
                    edge u-v);
        }
    }
}
```



a-b — 1
b — a — 1
a — c — 2
c — a — 2
b — c — -3
c — b — -3
c — d — 3
d — c — 3
b — d — 4
d — b — 4



complexity (V*E) space (n²)

Back Tracking

⇒ General technique not algorithm

⇒ finds a solution with n-tuples $\{x_1, x_2, x_3 \cdots x_n\}$

x_i = part of a set s_i.

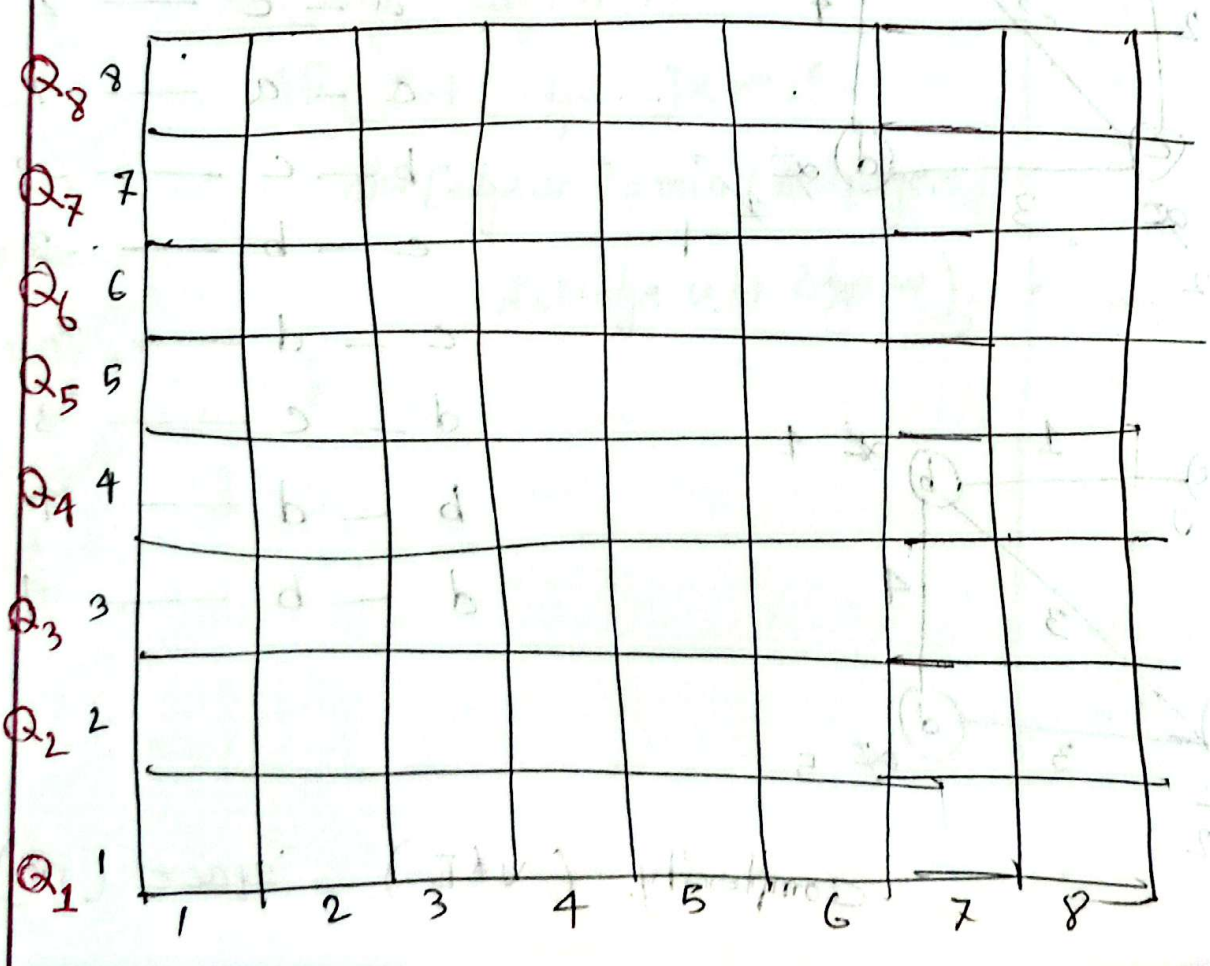
a objective function $P(x_1, x_2, x_3 \cdots x_n)$

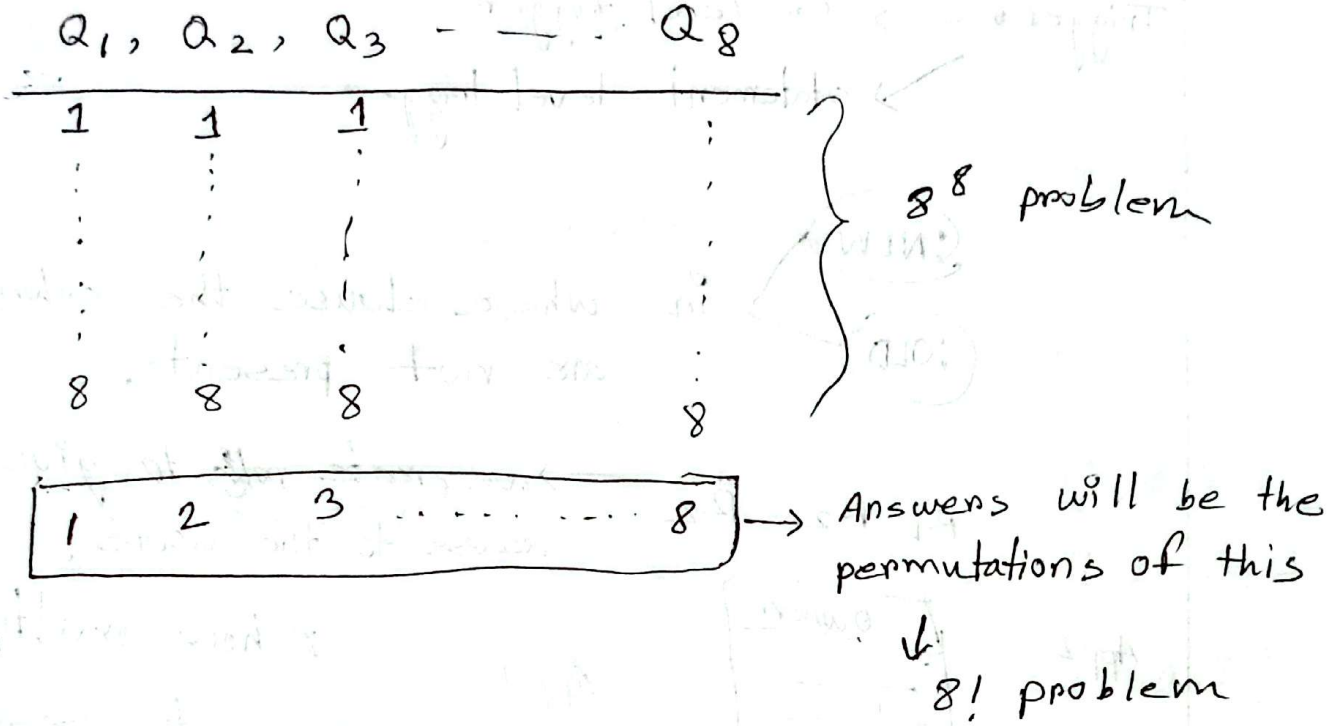
→ maximize/minimize

And some constraint on individual x_i

and or relations among $x_i, x_j, x_k \cdots$

The 8 queen problem:





Subset Sum:

$W = \{w_1, w_2, w_3 \cdots w_n\}$, Value v

find k tuples, $(x_1 \cdots x_k)$ $x_i \in W$ indexes

$\{j \mid j$ is a index in $w\}$

3 tuple $x_1 = 1, x_2 = 3, x_3 = 5$

constraint → $w_1 + w_3 + w_5 = V$

Explicit Constraints

30/7/25

Defines the constraints on individual x_i based on the problem.

Example: • The 8 Queen problem: $Q_i \in \{1, 2, \ldots, 8\}$

• $\boxed{l_i < x_i \leq u_i}$

Implicit Constraints: Constraints related to / among x_i.

Example:

• 8 Queen Problem: $Q_i \neq Q_j$ where $i \neq j$

$x_i \in S_i$, m_i is the size S_i

$(x_1, x_2, \ldots, x_n) \rightarrow m = m_1 m_2 m_3 \cdots m_n$

$m_1, m_2 \quad m_n$

for 8 Queen: $8, 8, \ldots, 8 \rightarrow 8^8$

$x_1 = \{1, 2, 3\}$

$x_2 = \{a, b\}$

$x_3 = \{x, y, z\}$

$[1, a, x]$

$[1, a, y]$

$1, a, z$

$1, b, x$

⋮

Solution Technique: The tuples/vectors are build by one by one variables and uses a modified $P_i(x_1, x_2, \ldots, x_i)$ to test if the subpart $(x_1, x_2, \ldots, x_i)$ Leads to a valid solution.

$P_i(x_1, x_2, \ldots, x_i)$ I have found it doesn't lead to a valid solution then I will backtrack. I will not assign values to $x_{i+1} \cdots x_n$.

This will save $(m_{i+1} m_{i+2} \cdots m_n)$ number of searching tuples.

Live node: all children not expanded (visited) yet.

Dead node: all children of a node expanded

Kill node: when a node violates a constraint, its children not expanded

Algorithm

$x_i \in S_i$, m_i is the size S_i

$(x_1, x_2 \cdots x_n)$

$m_1 \quad m_2 \quad \cdots \quad m_n \rightarrow m = m_1 m_2 \cdots m_n$

for 8 q → $8 \quad 8 \quad \cdots \quad 8 \rightarrow 8^8$

$x_1 = \{1, 2, 3\}$

$x_2 = \{a, b\}$

$x_3 = \{x, y, z\}$

[1, a, x]

[1, a, y]

[1, a, z]

[1, a, z]

[1, b, x]

<u>solution technique</u>:

The tupples/vectors are build by one by one variables and uses a modified $P_i(x_1, x_2, \cdots x_i)$ to test, if the subpart $(x_1, x_2 \cdots x_i)$ leads to a valid solutions.

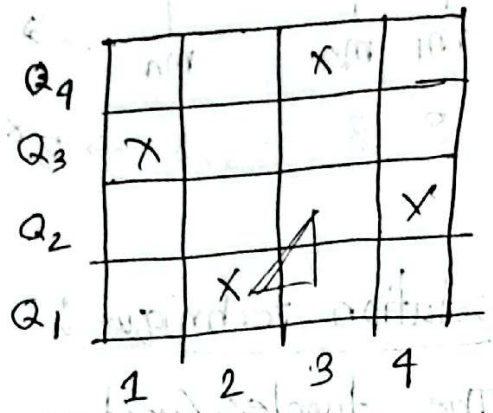
$P_i(x_1, x_2 \cdots x_i)$ I have found if doesn't lead to a valid solution then I will backtrack. I will not assign values to $x_{i+1} \cdots x_n$.

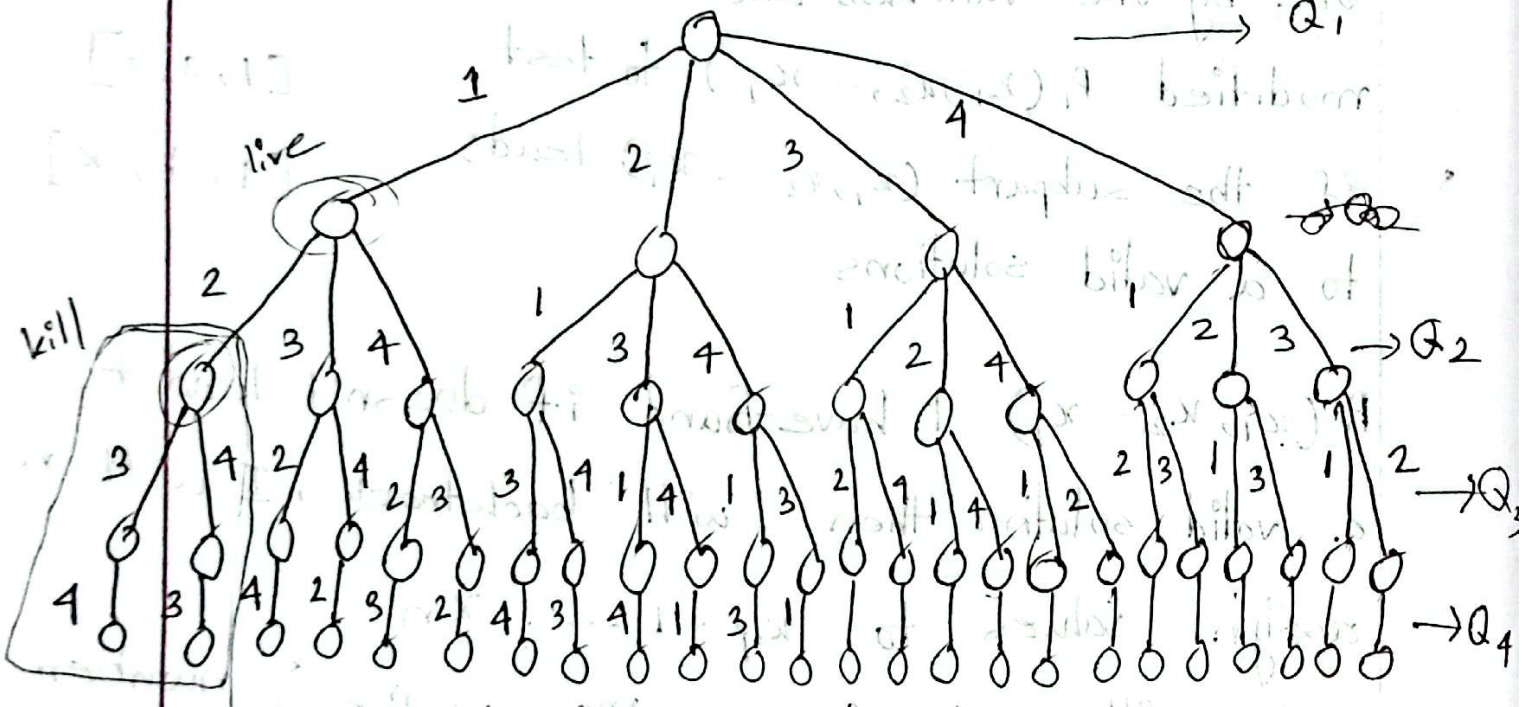
This will save $(m_{i+1}, m_{i+2} \cdots m_n)$ number of searching tupples.

Tree Organization :

The 4 Queen problem :

if Queens are set diagonally they can attack each other.





we will expand the tree starting from root until we violate the constraint.

live node -

dead node -

kill node -

$Q_1 \to 1 \to$ can't place Q_2 on 2 $\to$ kill node

$Q_1 \to 1 \to Q_2 \to 3 \to$ can't place Q_3 on 2 or 4

$\to$ dead node

$Q_1 \to 1 \to Q_2 \to 3 \to Q_3 \to 2 \to$ can't place $Q_4 \to 3$

at last $Q_1 \to 1$ would be a dead node

solution using recursive dfs.

$|Q_i - Q_j| \neq |i - j|$

Q_i: column number; i: Row number

#subset sum, coin change

Algorithm

Subset Sum

$(w_1, w_2, w_3 \ldots . w_n)$ and m. Goal is to find a subset such that there sum equal to m.

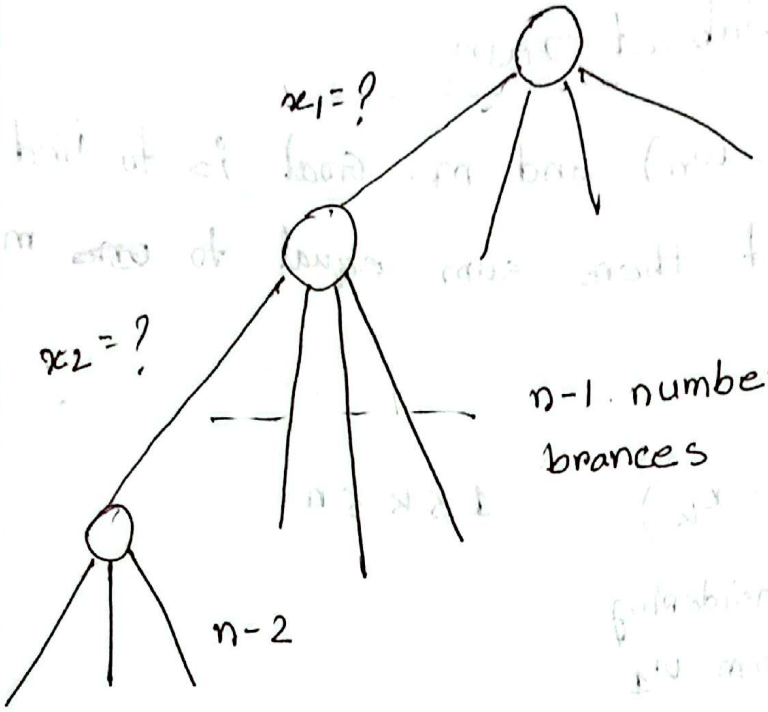
Formulation :

$\Rightarrow (x_1, x_2 \ldots . x_k)$ $\quad 1 \le k \le n$

x_1 : we are considering the value from w_1

explicit constraint : $x_i \in \{1 \ldots . n\}$ indicating the index of value in the set.

Implicit " : $\boxed{\sum_{i=1}^{k} w_{x_i} = m}$; $x_i \neq x_j$ when $i \neq j$ $\quad 1 \le i, j \le k$



at most n but in searching it will stop at k

we can get same set more than once.

$\{1, 3, 4\} \rightarrow \{3, 1, 4\}$

so, we can add another implicit constrain

$$x_i < x_{i+1}$$

this constraint is sufficient for $x_i \neq x_j$.

Formulation (2):

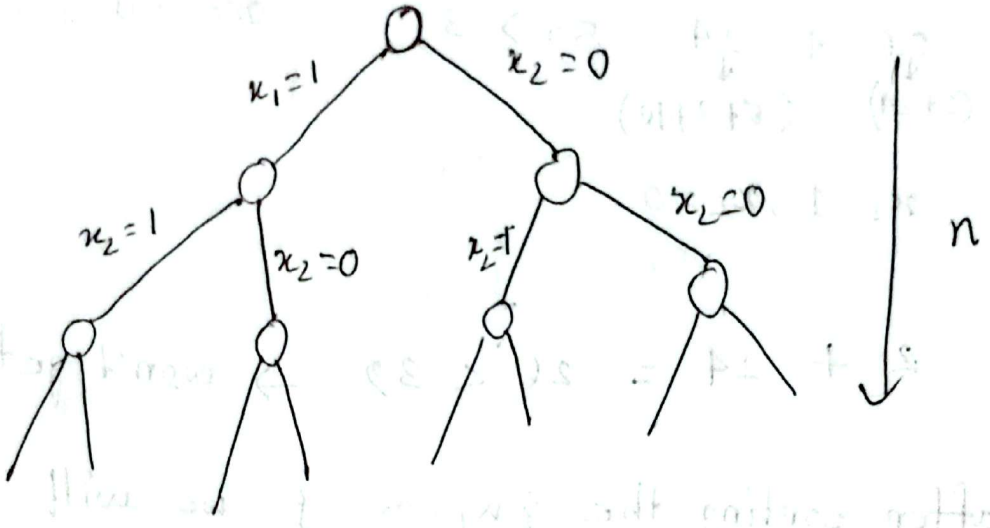
$\rightarrow (x_1, x_2 \cdots x_n)$ n-tuple.

$x_i = \{w_i, 0\} \rightarrow x_i = \{1, 0\}$ we can take the w_i(1) or we can not pick w_i(0)

$$\sum_{i=1}^{n} x_i = m \qquad \sum_{i=1}^{n} w_i x_i = m$$

Explicit : $x_i = \{1, 0\} \quad 1 \le i \le n$;

Implicit : $\sum_{i=1}^{n} w_i x_i = m$



Bounding function :

$B_k(x_1 \dots\dots x_k) \rightarrow$

$$\sum_{i=1}^{k} x_i w_i + \sum_{i=k+1}^{n} w_i \ge m$$

→ If it's true than the assignment may lead to a valid soln. otherwise we won't get valid soln. so, then backtrack

[2, 9] [9, 10, 21] 20 39

{ 2, 24, 5, 9, 10 }

k=2

$(x_1=1, x_2\ 1)$ $m = 39$

নেওয়া-না নেওয়া বোঝাতে 0, 1 দ্বারা

$26 + 24 = 50 \geq 39$ one sol^n: 24+5+10

(2+24) (5+9+10)

$x_1 = 1, x_2 = 0$

$2 + 24 = 26 < 39 \rightarrow$ won't get sol^n

after sorting the $\{w_1, w_2 \ldots\}$ we will add the constraint $\forall [w_i \leq w_{i+1}]$

also adding the condition

$\sum_{i=1}^{k} w_i x_i + w_{1+k} > m \rightarrow$ if it's true we will backtrack cause it won't give us any valid sol'n

Algorithm :

```
subsetSum (k, W, n) {
    x[1---n] variables
    if (k = n) print (x[1---n]);
      x[k] = 1;
    if (Bk (k) = true ){
      subsetSum (k+1, W, n);
    }
    else
      x[k] = 0;
      subsetSum (k+1, W, n);
}
```

$2^n \times n \rightarrow O(n) \rightarrow n2^n$

↓ ↓

binary tree — boundary function

Algorithm

Algebric Problems

Manipulation of ~~system~~ symbolic math like polynomial in a computer system or math expressions.

=> How they are represented

=> How some operations / computations are done on them

An Input $I \in S_1$, Goal is to calculated $F(I)$

where $F(I) \in S_1$

For efficiency I is transformed into $I' \in S_2$

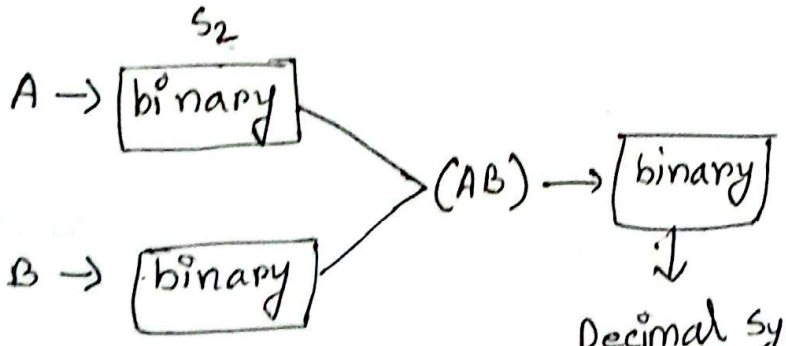
Apply $F(I') \in S_2$

Inverse the transformation $f(I')$ back to S_1

Example : Integer number manipulations

$A = 1234$, $B = 345$, $\in$ Decimal system (S_1)

$F(A,B) = AB$ ~~oo~~ multiplication



1234 — ndigit
345 — mdigit

1234 x 5
1234 x 40
1234 x 300

Add.

mul → (nm)

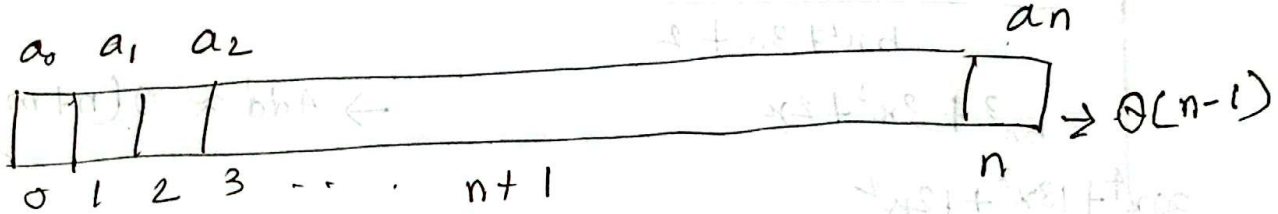
Add → m

example : Manipulation of polynomial

$$A(x) = \underbrace{a_n x^n}_{\text{highest degree}} + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} \ldots . a_1 x + a_0$$

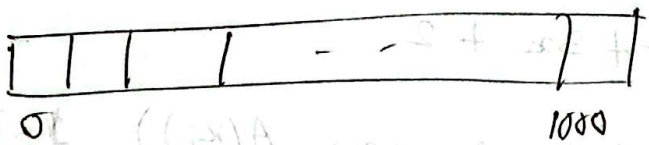
n - degree polynomial

n - degree bounded polynomial - degree $A(x) \leq n$



A(5) = ?

$A(x) = x^{1000} + 1$



→ for sparse poly array would have many empty index.

↳ instead of this we can use array of-tuples.

$[(n, a_n), \ldots . (n-5, a_5), \ldots . (0, a_0)] \rightarrow O(2n+2)$

[(1000,1), (0,1)] = 4 element

Addition : $A(x) = 5x^2 + 3x + 2$

$B(x) = 6x^2 + x + 1$

$11x^2 + 4x + 3$ $\rightarrow O(n+1)$

Multiplication : $A(x)\,B(x)$ $\quad O((n+1)(m+1))$

$\approx O(n^2)$

$5x^2 + 3x + 2$

$6x^2 + x + 1$

$5x^4 + 3x + 2$

$5x^3 + 3x^2 + 2x$

$30x^4 + 18x^3 + 12x^2$ $\rightarrow$ Add $\approx O(n+m)$

for multiplication best complexity is $\rightarrow n\log(n)$

point based polynomial representation ;

$A(x) = 5x^2 + 3x + 2$

n - point representation : $(x_i, A(x_i))$ $\quad 1 \le i \le n$

$(1, A(1)), (2, A(2))$

$a_2 1 + a_1 1 + 2 = 10$

$a_2 . 2^2 + a_1 . 2 + 2 = 28$

$A(x) = 5x^2 + 3x + 2$

a_2 a_1 a_0

we can represent in (n+1) point

$x_0 = 0$, $A(0) = 2$

$x_1 = 1$, $A(1) = 10$

$x_2 = 2$, $A(2) = 28$

$\{(0,2), (1,10), (2,28)\}$

Ax

from this we can get the polynomial

$a_2 \times 0 + a_1 \times 0 + a_0 = 2$

$a_2 \times 1^2 + a_1 \times 1 + a_0 = 10$

$a_2 \times 2^2 + a_1 \times 2 + a_0 = 28$

n^3 complexity.

$A(x)$ $B(x)$

$\{(0,2), (1,10), (2,28)\}$

$\{(0,1), (1,8), (2,27)\}$

$\{(0,2), (1,80), (2, 28 \times 27)\}$

we can not get $A(x)B(x)$ from this because we have 3 element but $A(x)B(x)$ will be a x^4 highest power poly.

So, represent $A(x)$, $B(x)$ with $(2n+1)$ element.

If we use fourier transform complexity → $n \log n$

Algorithm

$$A(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \ldots a_1 x + a_0$$

$$= \sum_{i=0}^{n-1} a_i x^i$$

n complex nth roots of unity

$$\omega_n^0, \omega_n^1, \omega_n^2 \ldots \omega_n^{n-1}$$

so n point representation of $A(x)$

$$\{(\omega_n^0, A(\omega_n^0)), (\omega_n^1, A(\omega_n^1)), \ldots (\omega_n^{n-1}, A(\omega_n^{n-1}))\}$$

steps of polynomial Multiplication using FFT.

1. Evaluate polynomials : $A(x)$, $B(x)$ on $2n$ points where $2n$ is a power of 2 and at least as large as degree $(A(x)$ + degree $(B(x)) \longrightarrow$ $\Theta(n \log n)$

2. Point wise multiplication of those $2n$ points of $C(x) = A(x)B(x) \longrightarrow \Theta(n)$

3. Inverse FFT on the point representation of $C(x) \longrightarrow \Theta(n \log n)$

Divide & Conquer Approach in FFT

① Evaluation

$$A(x) = \underbrace{a_0 + a_1 x + a_2 x^2 + a_3 x^3 \ldots + a_{n-1} x^{n-1}}_{n \text{ terms where } n \text{ is a power of } 2}$$

$$A_{even}(x) = a_0 + a_2 x^2 + a_4 x^4 + \ldots + a_{n-2} x^{n-2}$$

$$= a_0 + a_2 (x^2)^1 + a_4 (x^2)^2 + \ldots + a_{n-2} (x^2)^{\frac{n}{2}-1}$$

$$= A_{even}(x^2)$$

$$A_{odd} = a_1 x + a_3 x^3 + a_5 x^5 + \ldots + a_{n-1} x^{n-1}$$

$$= x(a_1 + a_3 (x^2) + a_5 (x^2)^2 + \ldots a_{n-1} (x^2)^{\frac{n}{2}-1}$$

$$= x A_{odd}(x^2)$$

$$A(x) = A_{even}(x^2) + x A_{odd}(x^2)$$

$$A(\omega_n^k) = A_{even}((\omega_n^k)^2) + \omega_n^k A_{odd}((\omega_n^k)^2)$$

$$= A_{even}(\omega_{n/2}^k) + \omega_n^k A_{odd}(\omega_{n/2}^k)$$

---- why aren't we usin horner's rule when time complexity is less than FFT ?

```
FFT(a,n){                          → complex unit = √-1
    ω_n = e^(2πi/n)                if (n == 1)
                                       return a;
```

$A(x) = a_0$

$a_{even} = \{a_0, a_2, a_4 \ldots a_{n-2}\}$

$a_{odd} = \{a_1, a_3, a_5 \ldots a_{n-1}\}$

$y_{even} = FFT(a_{even}, n/2)$

$y_{odd} = FFT(a_{odd}, n/2)$

for $k = 0$ to $n/2 - 1$ {

~~$y[k] = y_{even}[k] + \omega_n y_0$~~

$y[k] = y_{even}[k] + \omega \cdot y_{odd}[k]$

$y[n/2 + k] = y_{even}[k] - \omega\, y_{odd}[k]$

$\omega = \omega \cdot \omega_n$

}

return y

}

$\omega_n^0, \omega_n^1, \omega_n^2$

$k=0, k=1, k=2$

$\ldots \omega_n^{n-1}$

$k = n-1$

complexity $\rightarrow n \log n$

Algorithm

DFT – Discrete Fourier Transform

Interpolation using FFT :

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n^1 & (\omega_n^1)^2 & (\omega_n^1)^3 & (\omega_n^1)^4 & \cdots & (\omega_n)^{n-1} \\ 1 & \omega_n^2 & (\omega_n^2)^2 & (\omega_n^2)^3 & (\omega_n^2)^4 & \cdots & (\omega_n^2)^{n-1} \\ \vdots & & & & & & \\ 1 & \omega_n^{n-1} & (\omega_n^{n-1})^2 & (\omega_n^{n-1})^3 & (\omega_n^{n-1})^4 & \cdots & (\omega_n^{n-1})^{n-1} \end{bmatrix}$$

V_n

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

y

$\therefore y = V_n a$

$\therefore a = V_n^{-1} y$

$Y = DFT(a)$

$a = DFT(Y)$

a

$V_n[j,k] = \omega_n^{jk}$

for $j = 0, 1, 2, \ldots n-1$

$k = 0, 1, 2, \ldots n-1$

$V_n^{-1}[j,k] = \omega_n^{-jk/n}$

for $j = 0, 1, 2, \ldots n-1$

$k = 0, 1, 2, \ldots n-1$

Proof:

$$A(x) = a_0 + a_1 x + a_2 x^2 + \cdots a_{n-1} x^{n-1}$$

$$= \sum_{i=0}^{n-1} a_i x^i$$

$$y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j (\omega_n^k)^i = \sum_{i=0}^{n-1} a_i \omega_n^{ik}$$

$$y_0 = A(\omega_n^0)$$

$$y_1 = A(\omega_n^1)$$

$$\left| \{ (\omega_n^0, y_0), (\omega_n^1, y_1) \cdots (\omega_n^{n-1}, y_{n-1}) \} \right.$$

Proof:

$$V_n V_n^{-1} = I_n$$

$$V_n V_n^{-1}[j,k] = \sum_{i=0}^{n-1} V_n[j,i] \times V_n^{-1}[i,k]$$

$$= \sum_{i=0}^{n-1} \omega_n^{ji} \times \omega_n^{-ik} / n$$

$$= \frac{1}{n} \sum \omega_n^{(ji - ik)}$$

$1 + \omega + \omega^2 = 0$

$$V_n V_n^{-1}[j,k] = \frac{1}{n} \sum (\omega_n^{(j-k)})^i$$

$j \neq k$

$$= \frac{1}{n} \sum_{i=0}^{n-1} (\omega_n^{j-k})^i = 0$$

Summation Lemma

$j = k$

$$\frac{1}{n} \sum_{i=0}^{n-1} \frac{(\omega_n^0)^i}{1}$$

$$= 1$$

$$V_n^{-1} \quad \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n^{-1} & (\omega_n^{-1})^2 & \cdots & (\omega_n^{-1})^{n-1} \\ 1 & \omega_n^{-2} & (\omega_n^{-2})^2 & \cdots & (\omega_n^{-2})^{n-1} \\ \vdots & & & & \\ 1 & (\omega_n^{-(n-1)}) & (\omega_n^{-(n-1)})^2 & \cdots & (\omega_n^{-(n-1)})^{n-1} \end{bmatrix}$$

$$a_k = \frac{1}{n} \sum_{i=0}^{n-1} y_k (\omega_n^{-k})^i$$

$\omega = 1$

$\omega_n = e^{2\pi i/n}$

$\omega_n = e^{-2\pi i/n}$

```
FFT(a, n)
    if (n == 1) return 0;

    a_even = { a_0, a_2 .... a_{n-2} }

    a_odd = { a_1, a_3 .... a_{n-1} }

    y_even = FFT (a_even , n/2)

    y_odd = FFT (a_odd , n/2)

    w = 1 , w_n = e^{-2πi/n}

    for k = 0 .... (n/2 - 1)

        y_k = y_even[k] + w y_odd[k]

        y_{n/2 + k} = y_even[k] - w y_odd[k]

        w = w, w_n

    return y.
```

```
FFT^{-1}(y, n) {
    FFT(y, n)
    ↳ a
    for i = 0 ... n-1
        a = a/n
}
```